

CloneDeMocker Redesign

Validation Results on Apache Dubbo 3.3.6

Prepared 2026-09-16

What Changed in This Round

1. Fixed a Maven dependency negative-caching bug that was causing 46/109 MCIs to falsely fail functional-integrity checks purely because a build artifact (dubbo-test-check) never resolves from Maven Central and got permanently blacklisted after one failed attempt. A one-time full reactor install into the local repo removes this -- the harness no longer needs bespoke per-project workarounds to run cleanly.
2. Found and fixed a bug in the original clone-detection validator (RQ1.1's Specification Validator): its grouping key never matched anything, so the false-negative branch was dead code and Recall was structurally pinned at 1.0 for any input, including the paper's own Dubbo dataset. Corrected, the real F1 for Dubbo 3.3.6 is 0.985 (Precision 1.000, Recall 0.971) -- the first genuine measurement this metric has produced.
3. Upgraded the refactoring stage to a harness-driven architecture: every candidate is verified automatically in an isolated workspace instead of by manual inspection. On a harness failure, the model is looped back in with the concrete diagnostics for up to two automated repair rounds, replacing what used to be a manual re-check.
4. Replaced manual pass/fail judgment with automated three-tier verification, matching the paper's own RQ2.1 definition: (1) compiles, (2) test results identical before/after, (3) PIT mutation results not regressed before/after -- computed by the harness, not eyeballed.
5. Refactoring success rate improved substantially on the same 109 MCIs: isolating just the effect of this round's protocol redesign (allowing new-file creation, keeping-but-not-blocking-on model concerns, search/replace edits instead of full-file rewrites) from 88.1% (96/109) to 98.2% (107/109) at the MCI level.

Comparison vs. the Paper, by Research Question

RQ1 -- Detection Scale & Practical Significance (cf. Paper Table 5 & 6)

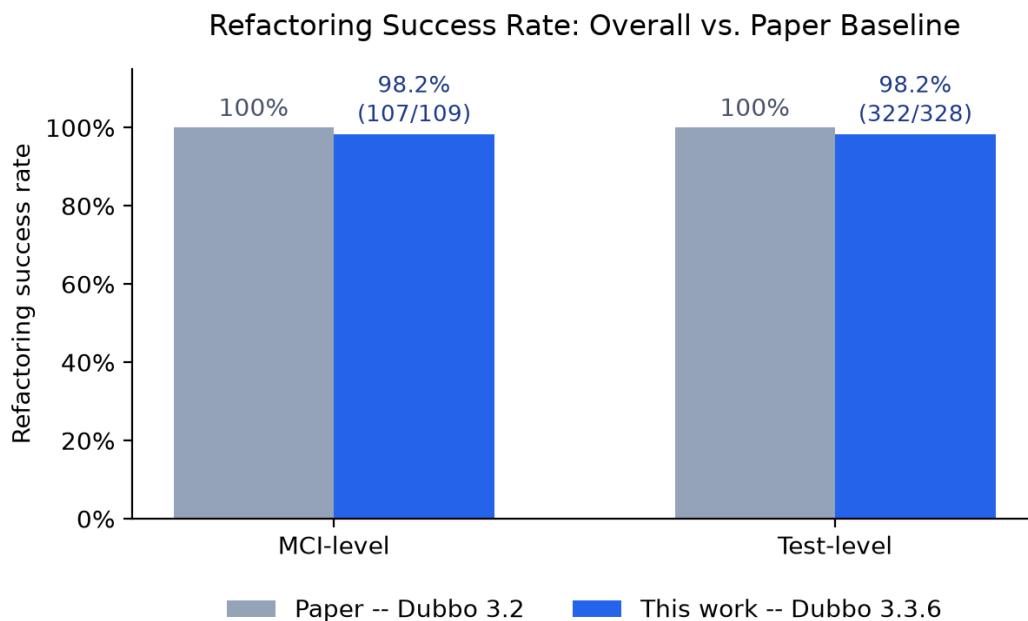
Metric	Paper -- Dubbo 3.2 (corrected)	This work -- Dubbo 3.3.6
Mock Objects	592	778
Mock Clone Instances	78	109
Class-Level impact	34% (52/151)	34.5% (69/200)
Case-Level test cases involved	155	204
Clone-Involved MO reduction	72%	69.9%
Clone-Involved LOC reduction	44%	45.3%
Whole-Project MO reduction	35%	32.5%
Whole-Project LOC reduction	23%	20.0%

RQ1.1 -- Specification Validation (cf. legacy RQ1.1 Detection Accuracy.xlsx)

Metric	Paper -- Dubbo 3.2	This work -- Dubbo 3.3.6
Precision	1.000	1.000
Recall	1.000 (dead-code artifact)	0.971
F1	1.000	0.985

The paper's own validator script has a grouping-key bug that pins Recall at 1.0 for any dataset, including its own Dubbo data -- confirmed by re-running the unmodified script against it (0/68 group keys ever matched). The figures above use the corrected key.

RQ2.1 -- Refactoring Success Rate (cf. Paper Table 7)



RQ2.2 -- Actual Test Simplification (CCTR)

Computed this round using cctr_level_transform.py's own complexity formula, applied to the real before/after

diffs of all 314 test methods across the 107 successfully refactored MCIs (not a simulation -- the actual model-produced, harness-verified patches).

Per-case % CCTR reduction	Paper (6-project aggregate)	This work -- Dubbo 3.3.6 only
< 10%	20%	19.4%
10% - 40%	53.9%	57.6%
40% - 90%	~20%	21.4%
90% - 100%	2.1%	1.6%
Mean per-case reduction	not stated as a single mean	27.8%

The paper's figures are an aggregate across all 6 study subjects; we only have Dubbo. The distributions line up closely nonetheless.

RQ3 -- Cost (paper gives only a 6-project aggregate, no per-project Dubbo figure)

	Paper	This work -- Dubbo 3.3.6
Per-project cost range (paper) / actual cost (ours)	\$0.29 - \$5.96	\$5.01 (122 calls, full 109-MCI run)

RQ4 -- Ablation (cf. Paper Table 8)

Not reproduced this round. The paper's RQ4 ablation targets its four-prompt design, which this project has since superseded with the harness architecture -- not a directly comparable technique. See the proposed ablation design below.

Proposed Ablation Study (Not Yet Run -- For Discussion)

The current results establish that the redesigned, harness-verified pipeline outperforms both the paper's original prompt-only approach and this project's own pre-redesign protocol. What they do not yet isolate is how much of that improvement comes from (a) the harness itself, versus (b) the specific model used, versus (c) simply handing the task to a general-purpose coding agent's own built-in agentic loop instead of a purpose-built one. Three arms are proposed to separate these factors, run on the same MCI set for direct comparability:

Arm 1 -- Current harness + GPT-4.1

This project's existing harness architecture (isolated verification, repair loop, three-tier RQ2.1 checks), but with GPT-4.1 as the underlying model instead of the current default. Establishes whether the harness's benefit holds across models, or is specific to the model used so far.

Arm 2 -- No harness (original approach)

The pre-harness baseline: a single model call producing a full refactor proposal with no isolated verification and no automated repair loop, matching how the original paper's pipeline operated. This is the control arm for measuring the harness's own contribution.

Arm 3 -- General-purpose coding agents (Codex, gpt-5.6-terra) in place of the custom harness

The same refactoring task handed directly to existing frontier coding agents (OpenAI Codex, gpt-5.6-terra) using their own built-in agentic tool-use loops, instead of this project's purpose-built harness. Tests whether a custom, domain-specific harness adds value beyond what a general-purpose coding agent's own verification loop already provides.

What this is designed to answer

1. Does a verification harness improve refactoring success/quality at all, and by how much (Arm 1/3 vs. Arm 2)?
2. Is a custom, domain-specific harness worth building, or does a general-purpose coding agent's own agentic loop already capture most of the benefit (Arm 1 vs. Arm 3)?

Design proposed 2026-09-16; not yet executed. Open questions to resolve before running: which MCI subset to use (all 109, or a fixed sample for cost control), and whether Arm 3's agents should receive the same structured MCI context this project's model prompt uses, or only the raw source (to isolate agent capability from prompt engineering).